
DSC 40B - Discussion 03

Problem 1.

- a) Here is an alternative implementation of the `binary_search` function from lecture.

```
def binary_search(arr, target, start=0, end=None):
    if end is None:
        end = len(arr)
    if start >= end:
        return False
    middle = int(ceil((start + end) / 2))
    if arr[middle] == target:
        return True
    elif arr[middle] < target:
        return binary_search(arr, target, middle + 1, end)
    else:
        return binary_search(arr, target, start, middle)
```

Termination and correctness?

Solution: The function will terminate since the size of the substrings is always lower than the input strings (and positive).

It is correct, because of the same reason as the original implementation.

- b) Here is an alternative implementation of the `factorial` function

```
def factorial(n):
    if n == 0:
        return 1
    return n * (n - 1) * factorial(n - 2)
```

Termination and correctness?

Solution: Assuming termination, the function is correct :

$$\text{factorial}(n) = n * (n - 1) * (\text{factorial}(n - 2)) \quad (1)$$

$$= n * (n - 1) * (n - 2)! \text{ (assuming correction before } n) \quad (2)$$

$$(3)$$

But the function does not terminate! If it is called on an odd number, it never reaches the $n = 0$ case.

Problem 2.

- a) State (but do not solve) the recurrence relation describing this function's run time.

```
import random
def foo(n):
    if n <= 2:
        return
```

```

for i in range(n):
    for j in range(i,n):
        print(i)
return foo(n//2) + foo(n//2)

```

Solution: $T(n) = 2T(n/2) + \Theta(n^2)$

- b) Suppose a binary search is performed on the following array using the implementation of *binary_search* from lecture. What is the worst case number of equality comparisons that would be made to search for an element in the array? That is, what is the greatest number of times that `arr[middle] == target` can be run?

[1, 4, 7, 8, 8, 10, 15, 51, 60, 65, 71, 72, 101]

Solution: 4.

The worst case number of comparisons occurs when what we're looking for is not in the array. Assume that the target is -999 so that every recursive call looks left (it actually matters whether we always look left or look right; if we look right, the number of comparisons will be one fewer). On the first call, **start** is 0 and the **stop** is 13. One comparison is made during this call to check if the middle element is the target.

On the second call, **start** is 0 and the **stop** is 6. One comparison is made during this call to check if the middle element is the target.

On the third call, **start** is 0 and the **stop** is 3. One comparison is made during this call to check if the middle element is the target.

On the fourth call, **start** is 0 and the **stop** is 1. One comparison is made during this call to check if the middle element is the target.

On the fifth call, **start** is 0 and the **stop** is 0. However, no comparisons between the target and other numbers are made during this call since the base case (any empty array) has been reached.

- c) State (but do not solve) the recurrence relation describing this function's run time.

```

def fibonacci(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fibonacci(n - 1) + fibonacci(n - 2)

```

How can we improve the run time of this function?

Solution: $T(n) = c + T(n - 1) + T(n - 2)$

to improve the run time, we can have the function output both the n th and $(n - 1)$ th fibonacci numbers, which means we only need one recursive call.

Problem 3.

What does this function do? What is its complexity?

```

def f(x, n):
    if n == 0:
        return 1
    y = f(x, n//2)
    if isEven(n):
        return y * y

```

```

else:
    return y * y * x

```

Solution: This function calculates x^n :

- Base case: $x^0 = 1$ OK
- Recursive case. let $H_n = "f(x, n) = x^n"$.
Assume $H_0, H_1, \dots, H_{n-1}$, prove H_n .
If n is even $f(x, n) = f(x, \frac{n}{2})f(x, \frac{n}{2}) = x^{\frac{n}{2}} * x^{\frac{n}{2}} = x^n$.
If n is odd $f(x, n) = f(x, \frac{n-1}{2})f(x, \frac{n-1}{2}) * x = x^{\frac{n-1}{2}} * x^{\frac{n-1}{2}} * x = x^n$.

Complexity: $T(n) = T(\frac{n}{2}) + c$ So $T(n) = \log_2(n)c$

Note that this is quite remarkable: The naive algorithm for computing x^n has $\Theta(n)$ complexity.

Problem 4.

Solve the following recurrence relations.

- a) $T(n) = T(n-1) + n$
 $T(0)=0$

Solution:

$$\begin{aligned}
 T(n) &= T(n-1) + n \\
 &= [T(n-2) + n-1] + n \\
 &= T(n-2) + 2n-1 \\
 &= [T(n-3) + n-2] + 2n-1 \\
 &= T(n-3) + 3n - (1+2) \\
 &= [T(n-4) + n-3] + 3n - (1+2) \\
 &= T(n-4) + 4n - (1+2+3)
 \end{aligned}$$

We can infer that $T(n) = T(n-k) + kn - \sum_{i=1}^{k-1} i$ in the k -th step.
 $T(0)$ is the base case. $n-k=0$ when $n=k$.

$$\sum_{i=1}^{n-1} i = \frac{(n-1)n}{2} = \frac{n^2 - n}{2}$$

$$\begin{aligned}
 T(n) &= T(n-n) + n \cdot n - \sum_{i=1}^{n-1} i \\
 &= T(0) + n^2 - \frac{n^2 - n}{2} \\
 &= 0 + n^2 - \frac{n^2 - n}{2} \\
 &= \theta(n^2)
 \end{aligned}$$

- b) $T(n)=4T(n/4) + n$
 $T(1)=1$

Solution:

$$\begin{aligned}T(n) &= 4 \cdot T(n/4) + n \\&= 4 [4 \cdot T(n/16) + n/4] + n \\&= 16 \cdot T(n/16) + 2n \\&= 16 [4 \cdot T(n/64) + n/16] + 2n \\&= 64 \cdot T(n/64) + 3n\end{aligned}$$

We can infer that in the k -th step, we have:

$$= 4^k \cdot T(n/4^k) + k \cdot n$$

The base case will be reached when $n/4^k = 1$, that is, when $k = \log_4 n$. Substituting this value of k into the general expression:

$$\begin{aligned}T(n) &= 4^{\log_4 n} \cdot T(n/4^{\log_4 n}) + n \cdot \log_4 n \\&= n \cdot T(n/n) + n \cdot \log_4 n \\&= n \cdot T(1) + n \cdot \log_4 n\end{aligned}$$

Since $T(1) = 1$, we have:

$$\begin{aligned}&= n + n \cdot \log_4 n \\&= \Theta(n \log_4 n)\end{aligned}$$

Since logarithms of different bases differ only by a constant factor, we typically omit the base when using asymptotic notation:

$$= \Theta(n \log n)$$

Problem 5.

Determine the recurrence relation describing the time complexity of each of the recursive algorithms below.

```
a) def fact(n):  
    if(n <= 1)  
        return 1  
    else  
        return n*fact(n-1)
```

Solution:

$$\begin{aligned}T(1) &= 1 \\T(n) &= T(n-1) + 1\end{aligned}$$

```
b) def max_arr(arr):  
    if(len(arr) == 1):  
        return arr[0]  
    mid = len(arr)//2  
    left_max = max_arr(arr[:mid])  
    right_max = max_arr(arr[mid:])  
    if(left_max > right_max):  
        return left_max
```

```
else:  
    return right_max
```

Solution:

$$T(1)=1$$

$$T(n)=2T(n/2)+n$$